SIMATS ENGINEERING

Department of Artificial Intelligence and Data Science

DSA04 – FUNDAMENTALS OF DATA SCIENCE

ASSIGNMENT 3 – CO3

EXPLORATORY DATA ANALYSIS AND DESCRIPTIVE STATISTICAL ANALYSIS USING PYTHON

Student Details	Information
Student Name	P V GIRIDHAR
Register Number	1972260022
Department	Artificial Intelligence and Data Science
Academic Year	2025 – 2029
Faculty Name	Dr Vanaja Ramalingam
Assignment Type	Individual
Duration	1 Week
Total Marks	50
Date of Submission	06 / 11 / 2026
Date of Evaluation	__ / __ / 2026

Mark Distribution

Task	Title	Maximum Marks
1	Data Preparation and Initial Exploratory Analysis	4
2	Data Summarization	4
3	Measures of Dispersion: Variance and Standard Deviation	5
4	Exploring Data Distribution	5
5	Outlier Detection and Treatment	5
6	Measuring Asymmetry: Skewness and Kurtosis	4
7	Continuous Distribution Analysis	4
8	Sampling and Estimation	4
9	Covariance and Correlation Analysis	8
10	Integrated Descriptive Statistics Case Study	7
	TOTAL	50

INTRODUCTION AND REPORT SCOPE

An engineering college wants to understand how its students are performing. Attendance, study hours, internal assessment marks, assignment scores and end-semester marks are recorded for every student, but raw numbers alone say very little. Descriptive statistics and exploratory data analysis (EDA) turn them into something readable: a typical value, how much students differ from it, what the distribution looks like, which records are unusual and which variables move together.

This report applies those techniques with Python (Pandas, NumPy, SciPy and Matplotlib) to a student-performance dataset. It follows the ten tasks in the assignment brief. Every task contains the Python code, a place for the output screenshot, and an interpretation written from the results.

Dataset used

A **sample dataset of 31 raw records (30 unique students)** was created for this assignment with the program `00_create_dataset.py` and saved as `students_raw.csv`. It is a hypothetical dataset prepared for learning. Students are identified only by coded IDs (S101 to S130), so no personally identifiable information is used. A fixed random seed is used, so the same file is produced every time the program is run. Two missing values and one duplicate record were included on purpose so that data preparation can be demonstrated, and two extreme internal-marks values were included so that outlier treatment can be demonstrated.

Column	Meaning	Type expected
Student_ID	Coded student identifier (S101 – S130)	Text
Department	CSE, ECE, MECH or AIDS	Text (category)
Attendance	Attendance percentage	Number
Study_Hours	Weekly self-study hours	Number
Internal_Marks	Internal assessment marks (out of 100)	Number
Assignment_Score	Assignment marks (out of 20)	Number
End_Sem_Marks	End-semester marks (out of 100)	Number

Data-quality feature in the raw file	Where it occurs
Duplicate record	S114 appears twice (31 rows, 30 unique students)
Missing values	Study_Hours: 1 (S110); Assignment_Score: 1 (S118)
Extreme internal marks	S107 has 12 and S122 has 100, while the class median is 68.5

Working assumptions

- Rows with the same Student_ID and identical values are duplicates; the first occurrence is kept.
- Missing values are filled with the **median** of the column, because the median is not pulled by extreme values.
- Extreme values are detected with the IQR rule. They may be genuine, so they are not deleted; the treatment used is stated in Task 5.
- Sample standard deviation and variance (divisor n − 1) are used throughout, because the 30 students are treated as a sample of the college.
- The threshold values used in the report (75% attendance, 10 to 18 study hours) are chosen for illustration and are stated where used. All data are hypothetical.

Program files and order of execution

Keep all program files in one folder, open each file in IDLE (File → Open) and press **F5** to run it. Run `00_create_dataset.py` first and `task1_prepare.py` second, because Task 1 produces `students_clean.csv`, which the programs for Tasks 2 to 9 read.

Order	Program file	Purpose	Needs
1	00_create_dataset.py	Creates students_raw.csv (run once)	–
2	task1_prepare.py	Task 1 – inspection and cleaning; saves students_clean.csv	students_raw.csv
3	task2_summary.py	Task 2 – summary statistics	students_clean.csv
4	task3_dispersion.py	Task 3 – variance and standard deviation	students_clean.csv
5	task4_distribution.py	Task 4 – histograms and box plots	students_clean.csv
6	task5_outliers.py	Task 5 – IQR outliers and capping	students_clean.csv
7	task6_skew_kurt.py	Task 6 – skewness and kurtosis	students_clean.csv
8	task7_continuous.py	Task 7 – continuous distribution of study hours	students_clean.csv
9	task8_sampling.py	Task 8 – sampling and estimation	students_clean.csv
10	task9_cov_corr.py	Task 9 – covariance, correlation, heatmap	students_clean.csv
11	task10_case_study.py	Task 10 – integrated analysis from raw data	students_raw.csv

Libraries required: pandas, numpy, matplotlib and scipy (install with: `pip install pandas numpy matplotlib scipy`). In newer Pandas versions text columns are shown with the type name `str` instead of `object`; both mean text.

Screenshot checklist

Each task has numbered boxes where the output screenshot is to be pasted. Take the screenshot after running the program in IDLE: the shell window for text output, and the plot window for figures.

No.	Task	Program	What to capture
1	Task 1	task1_prepare.py	IDLE shell output
2	Task 2	task2_summary.py	IDLE shell output
3	Task 3	task3_dispersion.py	IDLE shell output
4	Task 4	task4_distribution.py	Histogram window
5	Task 4	task4_distribution.py	Box plot window
6	Task 4	task4_distribution.py	IDLE shell output
7	Task 5	task5_outliers.py	IDLE shell output
8	Task 6	task6_skew_kurt.py	IDLE shell output
9	Task 7	task7_continuous.py	Distribution plot window
10	Task 7	task7_continuous.py	IDLE shell output
11	Task 8	task8_sampling.py	IDLE shell output
12	Task 9	task9_cov_corr.py	IDLE shell output
13	Task 9	task9_cov_corr.py	Heatmap window
14	Task 9	task9_cov_corr.py	Scatter plot window
15	Task 10	task10_case_study.py	IDLE shell output
16	Task 10	task10_case_study.py	Plot window (two plots)

TASK 1 – Data Preparation and Initial Exploratory Analysis (4 Marks)

Prepare the student dataset for descriptive statistical analysis.

1.1 Python code

The program loads the raw file, inspects it, then removes the duplicate and fills the two missing values with the column median. The cleaned file is saved for the later tasks.

Python code *(file: task1_prepare.py)*

```
# TASK 1 - Data Preparation and Initial Exploratory Analysis
import pandas as pd

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

df = pd.read_csv("students_raw.csv")

print("===== FIRST 5 RECORDS =====")
print(df.head())
print("\n===== DIMENSIONS (rows, columns) =====")
print(df.shape)
print("\n===== COLUMN NAMES =====")
print(df.columns.tolist())
print("\n===== DATA TYPES =====")
print(df.dtypes)

print("\n===== NUMERICAL VARIABLES =====")
print(df.select_dtypes(include="number").columns.tolist())
print("\n===== CATEGORICAL VARIABLES =====")
print(df.select_dtypes(exclude="number").columns.tolist())

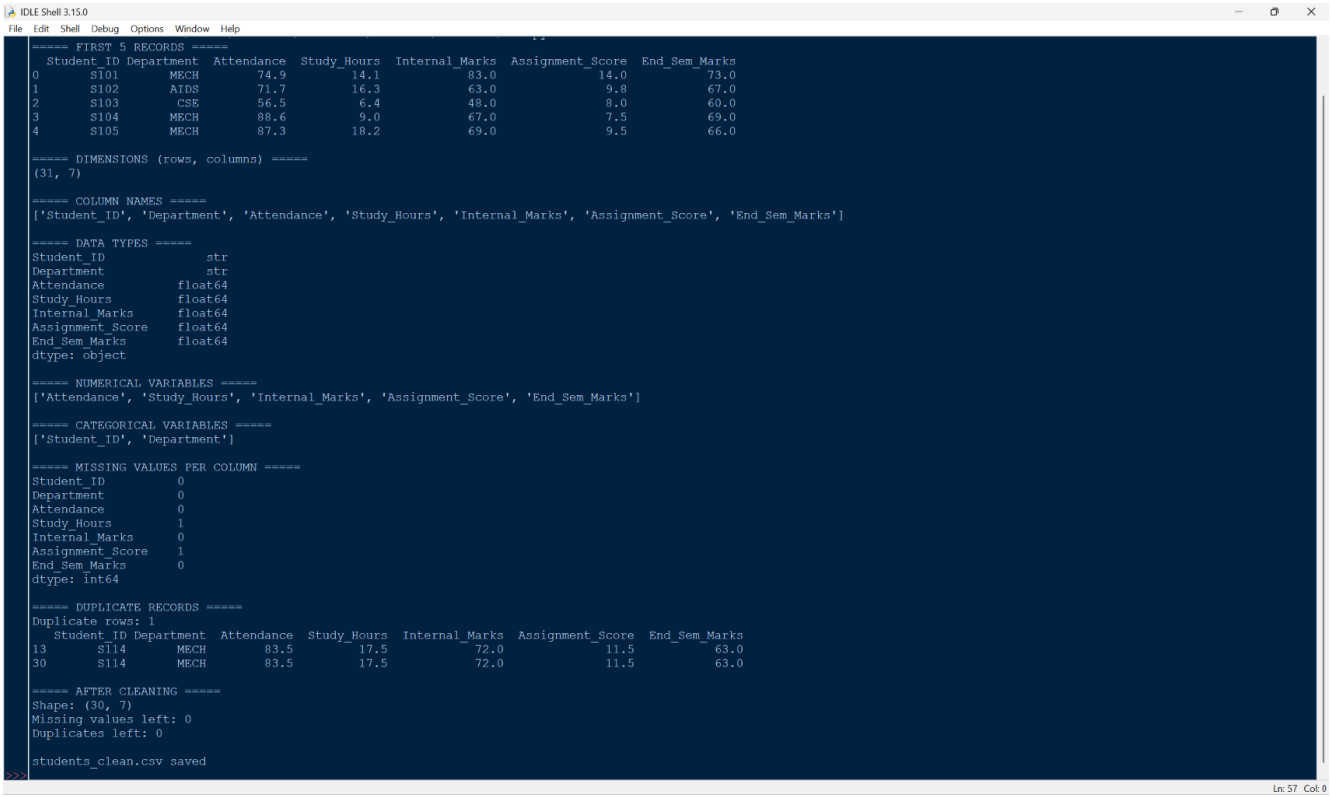
print("\n===== MISSING VALUES PER COLUMN =====")
print(df.isnull().sum())
print("\n===== DUPLICATE RECORDS =====")
print("Duplicate rows:", df.duplicated().sum())
print(df[df.duplicated(keep=False)])

# ---- cleaning ----
clean = df.drop_duplicates().copy()
for col in ["Study_Hours", "Assignment_Score"]:
    clean[col] = clean[col].fillna(clean[col].median())

print("\n===== AFTER CLEANING =====")
print("Shape:", clean.shape)
print("Missing values left:", clean.isnull().sum().sum())
print("Duplicates left:", clean.duplicated().sum())

clean.to_csv("students_clean.csv", index=False)
print("\nstudents_clean.csv saved")
```

Output – Screenshot 1



1.2 Interpretation of the initial inspection

Observation	Result from the inspection
Dimensions	31 rows × 7 columns (31 raw records for 30 unique students)
Column names	Student_ID, Department, Attendance, Study_Hours, Internal_Marks, Assignment_Score, End_Sem_Marks
Numerical variables	Attendance, Study_Hours, Internal_Marks, Assignment_Score, End_Sem_Marks (float64)
Categorical variables	Student_ID (identifier) and Department
Missing values	Study_Hours: 1 and Assignment_Score: 1
Duplicate records	1 (the last row repeats S114)

How missing values and duplicates affect descriptive analysis. Pandas skips missing values when it calculates a mean or standard deviation, so each column is summarised on a different number of students (29 instead of 30 here) and the summaries are not strictly comparable. In a larger dataset with more gaps, the result could also be biased if the missing entries are not random. The duplicate row counts S114 twice, which gives that student double weight in every mean, variance and count, and makes the dataset look bigger than it is.

Treatment used. `drop_duplicates()` removed the repeated S114 row (31 → 30 rows). The two missing values were replaced by the median of their column, which keeps all 30 students and is not distorted by extreme values. After cleaning, no missing values or duplicates remain.

Why data preparation is needed before calculating statistical measures. Every statistical measure is only as reliable as the data behind it. Duplicates inflate counts, missing values change the number of observations, wrong data types stop calculations, and impossible values distort the mean and spread. Preparing the data first

means that the mean, median, variance and correlations calculated in the remaining tasks describe the real student group and not errors in the file.

TASK 2 – Data Summarization (4 Marks)

Summarise the numerical variables using measures of central tendency and range.

2.1 Python code

Attendance, internal marks and study hours are summarised. `describe()` then gives the full descriptive summary, and the mean and median of internal marks are compared.

Python code *(file: task2_summary.py)*

```
# TASK 2 - Data Summarization
import pandas as pd

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

df = pd.read_csv("students_clean.csv")
cols = ["Attendance", "Internal_Marks", "Study_Hours"]

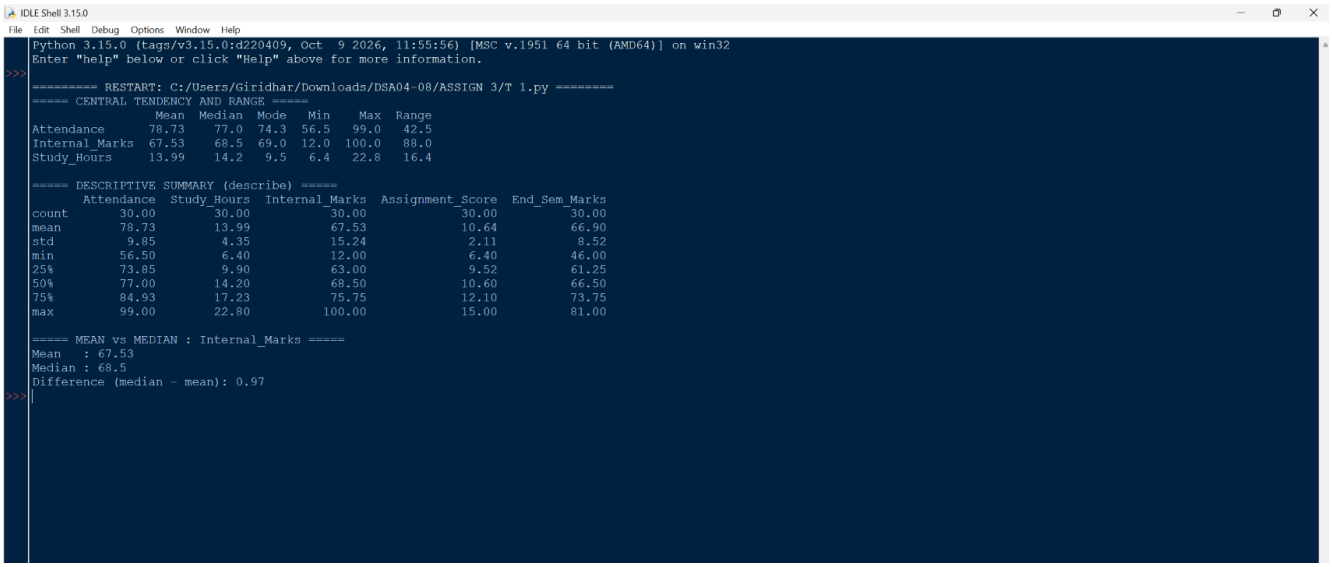
table = pd.DataFrame({
    "Mean": df[cols].mean(),
    "Median": df[cols].median(),
    "Mode": [df[c].mode()[0] for c in cols],
    "Min": df[cols].min(),
    "Max": df[cols].max(),
})
table["Range"] = table["Max"] - table["Min"]

print("===== CENTRAL TENDENCY AND RANGE =====")
print(table.round(2))

print("\n===== DESCRIPTIVE SUMMARY (describe) =====")
print(df.describe().round(2))

mean_im = df["Internal_Marks"].mean()
med_im = df["Internal_Marks"].median()
print("\n===== MEAN vs MEDIAN : Internal_Marks =====")
print("Mean   :", round(mean_im, 2))
print("Median :", round(med_im, 2))
print("Difference (median - mean):", round(med_im - mean_im, 2))
```

Output – Screenshot 2



2.2 Calculation table

Variable	Mean	Median	Mode	Min	Max	Range
Attendance (%)	78.73	77.00	74.3*	56.5	99.0	42.5
Internal_Marks	67.53	68.50	69.0	12.0	100.0	88.0
Study_Hours	13.99	14.20	9.5*	6.4	22.8	16.4

**More than one mode exists (Attendance: 74.3, 83.5 and 99.0 each occur twice; Study_Hours: 9.5 and 14.2 each occur twice). Pandas lists them in order and the table shows the first one. For these two variables the mode is not a useful summary. For Internal_Marks the mode is clear: 69 occurs four times.*

2.3 Descriptive summary (describe)

Statistic	Attendance	Study_Hours	Internal_Marks	Assignment_Score	End_Sem_Marks
count	30	30	30	30	30
mean	78.73	13.99	67.53	10.64	66.90
std	9.85	4.35	15.24	2.11	8.52
min	56.50	6.40	12.00	6.40	46.00
25%	73.85	9.90	63.00	9.52	61.25
50%	77.00	14.20	68.50	10.60	66.50
75%	84.93	17.23	75.75	12.10	73.75
max	99.00	22.80	100.00	15.00	81.00

2.4 Mean versus median, and choice of measure

For Internal_Marks the mean is **67.53** and the median is **68.50**, so the median is about 0.97 marks higher. When the mean falls below the median, the distribution is usually pulled towards the low side by a few small values. Here the very low mark of 12 (S107) drags the mean down more than the high mark of 100 (S122) pushes it up, because 12 is much further from the centre (56.5 below the median against 31.5 above).

Most suitable measure: the median. Internal marks contain extreme values, and the median does not move when one or two values at the ends change, while the mean does. The median of 68.50 describes a typical student better than the mean. For attendance and study hours, where no strong outliers are present, the mean and median are close and either could be used.

TASK 3 – Measures of Dispersion: Variance and Standard Deviation (5 Marks)

Examine how much student performance varies.

3.1 Python code

`var()` and `std()` in Pandas use the sample formula (divisor n − 1). Because the three variables are on different scales (percentage, marks out of 100, marks out of 20), the coefficient of variation (CV = standard deviation ÷ mean × 100) is also calculated to make a fair comparison.

Python code *(file: task3_dispersion.py)*

```
# TASK 3 - Variance and Standard Deviation
import pandas as pd

pd.set_option("display.width", 200)

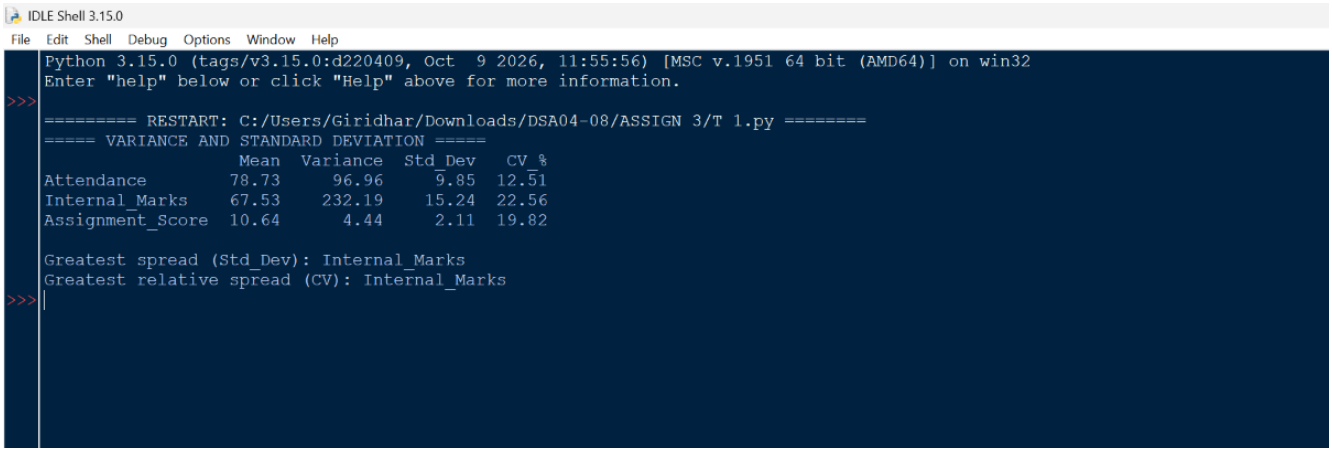
df = pd.read_csv("students_clean.csv")
cols = ["Attendance", "Internal_Marks", "Assignment_Score"]

out = pd.DataFrame({
    "Mean": df[cols].mean(),
    "Variance": df[cols].var(),          # sample variance (n - 1)
    "Std_Dev": df[cols].std(),
})
out["CV_%"] = out["Std_Dev"] / out["Mean"] * 100   # to compare different scales

print("===== VARIANCE AND STANDARD DEVIATION =====")
print(out.round(2))

print("\nGreatest spread (Std_Dev):", out["Std_Dev"].idxmax())
print("Greatest relative spread (CV):", out["CV_%"].idxmax())
```

Output – Screenshot 3



3.2 Output table

Variable	Mean	Variance	Std. deviation	CV (%)
Attendance	78.73	96.96	9.85	12.51
Internal_Marks	67.53	232.19	15.24	22.56
Assignment_Score	10.64	4.44	2.11	19.82

3.3 Interpretation

Comparison of two variables. Internal marks have a standard deviation of 15.24 against 9.85 for attendance, and a variance of 232.19 against 96.96, so internal marks are about 2.4 times as variable in terms of variance. Students differ more in their internal marks than in their attendance.

What a high or low standard deviation means here. A low standard deviation means that students are close to the average, so the group is fairly uniform and the average describes most students well. A high standard deviation means that students are spread widely, so some are doing much better or much worse than the average, and the average alone hides this. For a college, high spread in marks signals a need for different kinds of support (extra help for the weak, enrichment for the strong) rather than one approach for everyone.

Variable with the greatest variability: Internal_Marks. It has the largest standard deviation (15.24), the largest variance (232.19) and the largest CV (22.56%). Assignment scores come second on relative spread (19.82%), which shows that even a small standard deviation of 2.11 is meaningful when the scores are out of 20. Attendance is the most consistent variable (12.51%). Part of the spread in internal marks comes from the two extreme values, 12 and 100, which are examined in Task 5.

TASK 4 – Exploring Data Distribution (5 Marks)

Study how attendance and internal marks are distributed using histograms and box plots.

4.1 Python code

Python code *(file: task4_distribution.py)*

```
# TASK 4 - Exploring Data Distribution
import pandas as pd
import matplotlib.pyplot as plt

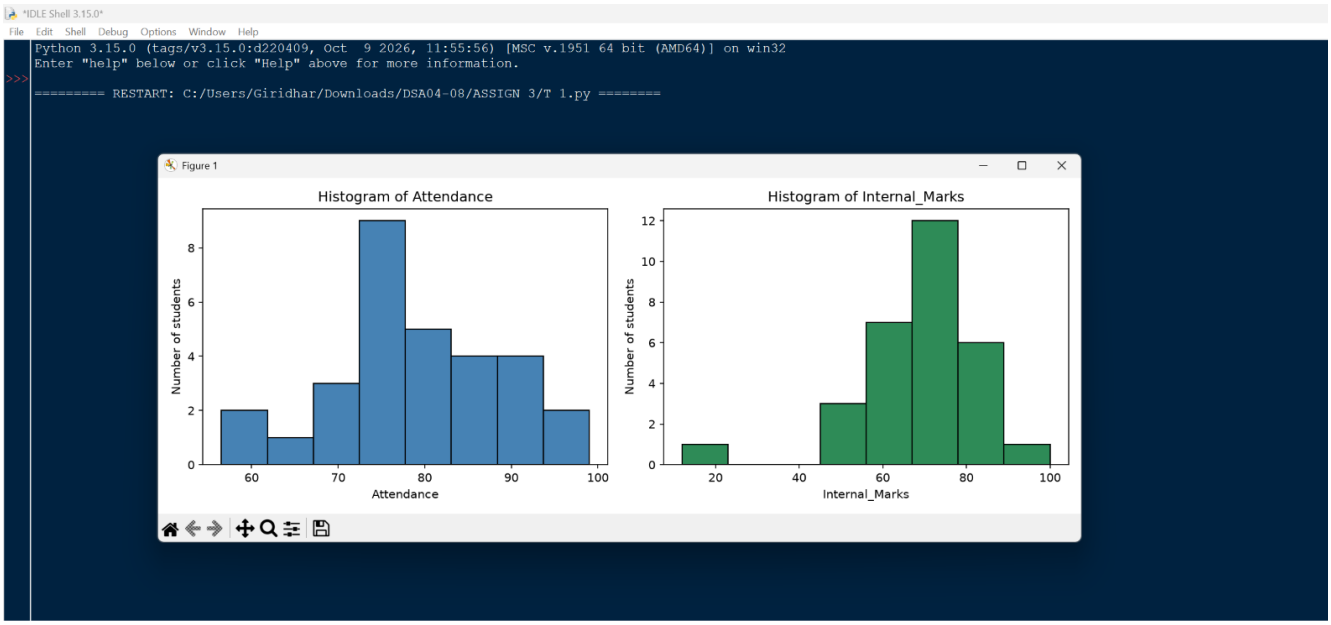
df = pd.read_csv("students_clean.csv")
cols = ["Attendance", "Internal_Marks"]

# histograms
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
for a, c, colour in zip(ax, cols, ["steelblue", "seagreen"]):
    a.hist(df[c], bins=8, color=colour, edgecolor="black")
    a.set_title(f"Histogram of {c}")
    a.set_xlabel(c)
    a.set_ylabel("Number of students")
plt.tight_layout()
plt.savefig("task4_histograms.png", dpi=120)
plt.show()

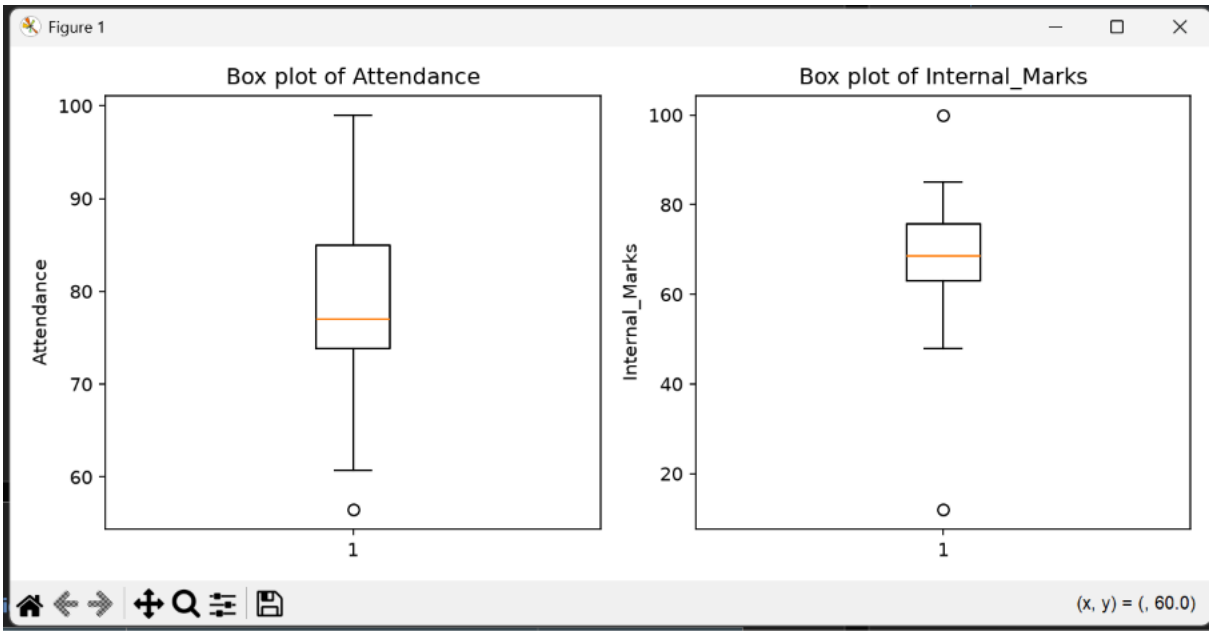
# box plots
fig, ax = plt.subplots(1, 2, figsize=(9, 4))
for a, c in zip(ax, cols):
    a.boxplot(df[c])
    a.set_title(f"Box plot of {c}")
    a.set_ylabel(c)
plt.tight_layout()
plt.savefig("task4_boxplots.png", dpi=120)
plt.show()

print(df[cols].describe().round(2))
```

Output – Screenshot 4



Output – Screenshot 5



4.2 Description of each distribution

Attendance. The histogram is roughly bell-shaped and nearly symmetric, with its tallest bar (9 students) at about 72 to 78%. The centre is close to 77 to 79% (median 77.0, mean 78.73). The spread is moderate: the standard deviation is 9.85 and the middle 50% of students lie between 73.85% and 84.93% (IQR 11.08). The box plot shows one point below the lower whisker, 56.5% (S103), which is a mild low outlier; the upper whisker is slightly longer than the lower one.

Internal marks. The histogram has a clear peak, with its tallest bar (12 students) at about 67 to 78 marks, and most students fall between 56 and 89 marks. The centre is about 68 (median 68.5). The middle 50% lie between 63.0 and 75.75 (IQR 12.75). Two isolated observations appear far from the rest: a single bar near 12 to 23 marks on the left, and 100 on the far right. The box plot shows these as two separate points, 12 (S107) and 100 (S122). The left side of the histogram is stretched out further than the right, so the shape is slightly skewed to the left.

4.3 Comparison

Feature	Attendance	Internal_Marks
Shape	Roughly symmetric, bell-shaped	Peaked, with a long left tail
Centre (median)	77.0	68.5
Spread (std. deviation / IQR)	9.85 / 11.08	15.24 / 12.75
Range	42.5 (56.5 to 99.0)	88.0 (12.0 to 100.0)
Extreme observations	1 mild low value (56.5)	2 extreme values (12 and 100)

The two distributions differ mainly in how they behave at the ends. The bulk of the internal marks is as concentrated as attendance (IQR 12.75 marks against 11.08 percentage points), yet the total range of internal marks is more than twice as wide because of the two extreme records. Attendance spreads out evenly on both sides, while internal marks are tightly packed around the middle with a few distant values. This is why the standard deviation of internal marks in Task 3 looked so high.

TASK 5 – Outlier Detection and Treatment (5 Marks)

Detect unusual observations in internal marks with the Interquartile Range (IQR) method and treat them.

5.1 Python code

Python code *(file: task5_outliers.py)*

```
# TASK 5 - Outlier Detection and Treatment (IQR method)
import pandas as pd

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

df = pd.read_csv("students_clean.csv")
x = df["Internal_Marks"]

q1, q3 = x.quantile(0.25), x.quantile(0.75)
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr

print("Q1 =", q1, "| Q3 =", q3, "| IQR =", iqr)
print("Lower bound =", lower, "| Upper bound =", upper)

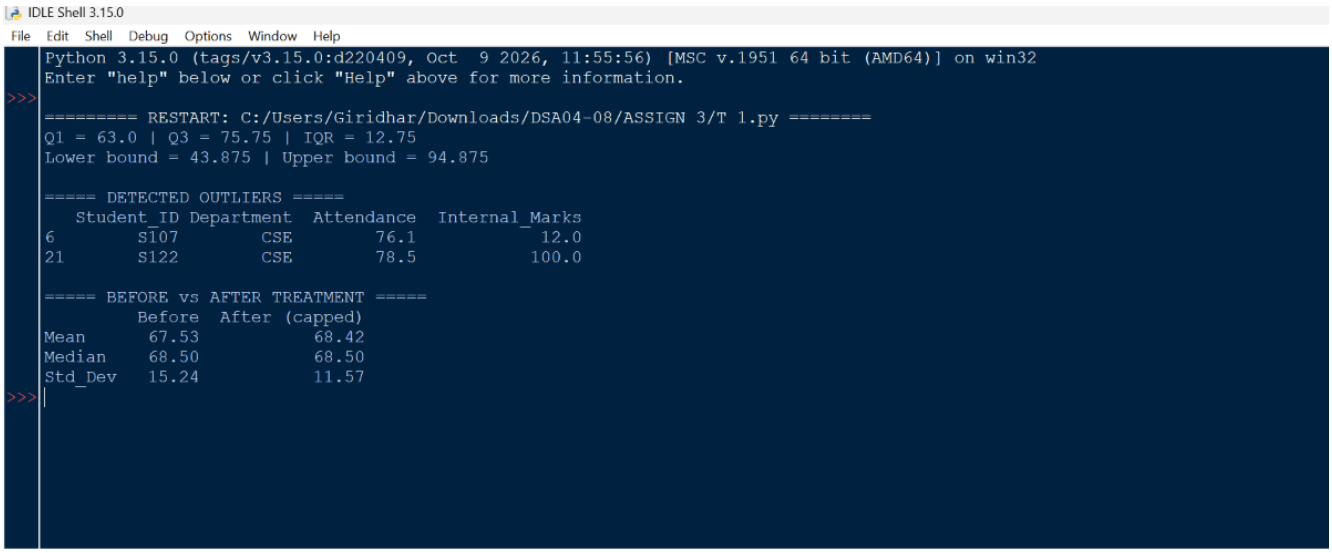
mask = (x < lower) | (x > upper)
print("\n===== DETECTED OUTLIERS =====")
print(df.loc[mask, ["Student_ID", "Department", "Attendance", "Internal_Marks"]])

# treatment: capping (winsorising) at the IQR bounds
capped = x.clip(lower, upper)

compare = pd.DataFrame({
    "Before": [x.mean(), x.median(), x.std()],
    "After (capped)": [capped.mean(), capped.median(), capped.std()],
}, index=["Mean", "Median", "Std_Dev"])

print("\n===== BEFORE vs AFTER TREATMENT =====")
print(compare.round(2))
```

Output – Screenshot 6



5.2 IQR calculations and bounds

Quantity	Formula	Value
Q1 (25th percentile)	–	63.00
Q3 (75th percentile)	–	75.75
IQR	Q3 − Q1	12.75
1.5 × IQR	1.5 × 12.75	19.125
Lower bound	Q1 − 1.5 × IQR = 63 − 19.125	43.875
Upper bound	Q3 + 1.5 × IQR = 75.75 + 19.125	94.875

Student	Department	Attendance	Internal marks	Reason
S107	CSE	76.1	12	Below the lower bound (43.875)
S122	CSE	78.5	100	Above the upper bound (94.875)

5.3 Treatment: capping at the IQR bounds

The treatment chosen is **capping** (also called winsorising): values below the lower bound are raised to 43.875 and values above the upper bound are lowered to 94.875, using `clip()`. So 12 becomes 43.875 and 100 becomes 94.875.

Statistic	Before treatment	After capping
Mean	67.53	68.42
Median	68.50	68.50
Standard deviation	15.24	11.57

5.4 Justification

- **Median unchanged, mean moved little.** The median stays at 68.50 and the mean changes by under one mark (67.53 → 68.42), so the typical value is not changed. The standard deviation falls by about 24% (15.24 → 11.57), which shows that the two extreme records were responsible for much of the apparent spread.
- **Why not remove them.** Only 30 students are available, so deleting two records would lose 7% of the data. Both students have ordinary attendance and end-semester marks (67 and 63), so the rest of their records is valid.
- **Why not simply retain them.** Leaving them in would keep the mean and standard deviation distorted, and would affect the correlations in Task 9.
- **Caution.** The marks could be real (a very strong or very weak performance) or data-entry errors. Capping is a safe middle course for the analysis, and the original values are kept in `students_clean.csv` so the records can be checked with the course faculty.

TASK 6 – Measuring Asymmetry: Skewness and Kurtosis (4 Marks)

Examine the asymmetry and tail behaviour of attendance and internal marks.

6.1 Python code

`scipy.stats.skew()` and `scipy.stats.kurtosis()` are used. SciPy reports **excess kurtosis** (Fisher definition), where a normal distribution scores 0. The calculation is repeated for internal marks after capping, to show how much the outliers matter.

Python code *(file: task6_skew_kurt.py)*

```
# TASK 6 - Skewness and Kurtosis
import pandas as pd
from scipy.stats import skew, kurtosis

df = pd.read_csv("students_clean.csv")

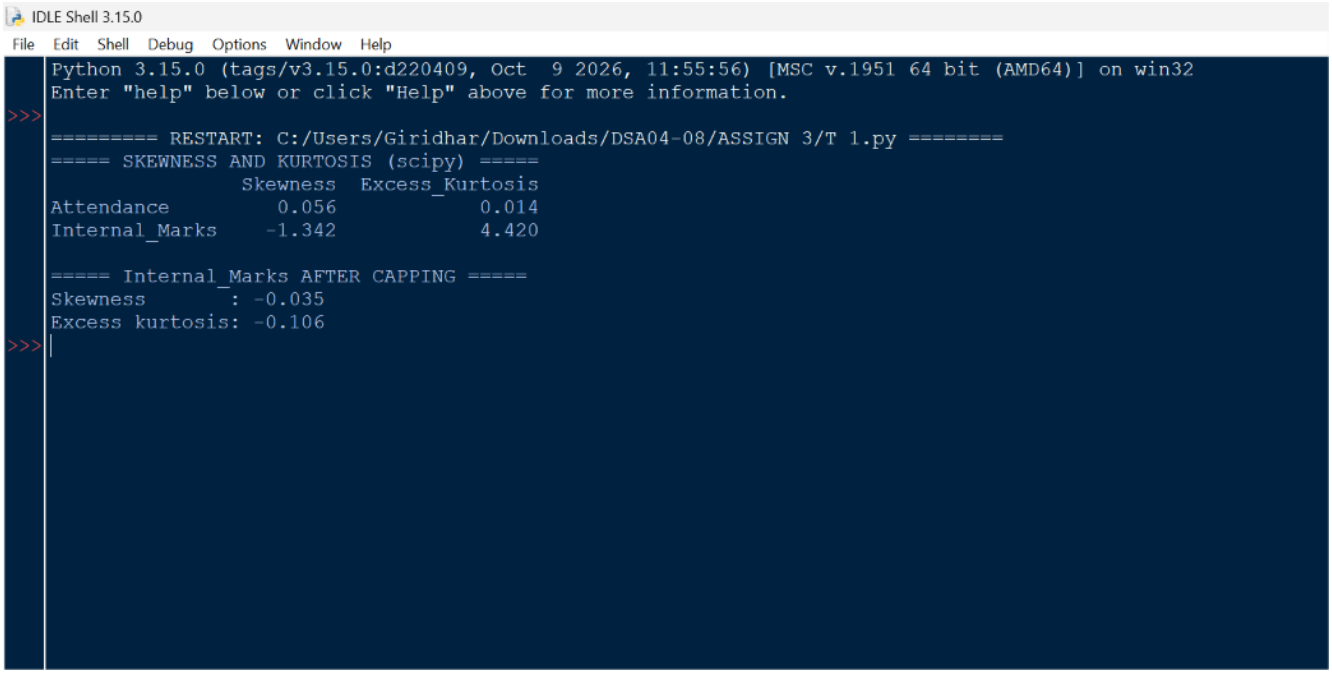
res = pd.DataFrame({
    "Skewness": [skew(df["Attendance"]), skew(df["Internal_Marks"])],
    "Excess_Kurtosis": [kurtosis(df["Attendance"]), kurtosis(df["Internal_Marks"])],
}, index=["Attendance", "Internal_Marks"])

print("===== SKEWNESS AND KURTOSIS (scipy) =====")
print(res.round(3))

# internal marks again after capping the two outliers
x = df["Internal_Marks"]
q1, q3 = x.quantile(0.25), x.quantile(0.75)
iqr = q3 - q1
capped = x.clip(q1 - 1.5 * iqr, q3 + 1.5 * iqr)

print("\n===== Internal_Marks AFTER CAPPING =====")
print("Skewness       :", round(skew(capped), 3))
print("Excess kurtosis:", round(kurtosis(capped), 3))
```

Output – Screenshot 7



6.2 Calculated values

Variable	Skewness	Excess kurtosis
Attendance	0.056	0.014
Internal_Marks	−1.342	4.420
Internal_Marks (after capping)	−0.035	−0.106

6.3 Interpretation

Skewness. Positive skewness means a longer tail on the right (a few unusually high values), negative skewness means a longer tail on the left (a few unusually low values), and a value near zero means the distribution is roughly symmetric. As a rough guide, values between −0.5 and 0.5 are treated as approximately symmetric.

Kurtosis. Excess kurtosis near zero (mesokurtic) means tails similar to a normal curve. A positive value (leptokurtic) means a sharper peak and heavier tails, so extreme values are more likely. A negative value (platykurtic) means a flatter distribution with lighter tails.

- **Attendance:** skewness 0.056 is almost zero and kurtosis 0.014 is almost zero. The distribution is symmetric and normal-like in its tails, which matches the bell-shaped histogram and the balanced box plot in Task 4.
- **Internal marks:** skewness −1.342 shows strong negative skew, and kurtosis 4.420 shows a very sharp peak with heavy tails. This matches the Task 4 plots: most marks are packed around 68, while the isolated bar near 12 pulls the left tail out. The mark of 12 has the larger effect, since it lies much further from the centre than 100 does.
- **After capping:** the skewness becomes −0.035 and the kurtosis −0.106, both close to zero. The strong asymmetry and heavy tails came almost entirely from two records, not from the whole group of students.

Pandas `skew()` and `kurt()` apply a small-sample correction, so they give slightly different numbers from SciPy's default; the conclusions are the same.

TASK 7 – Continuous Distribution Analysis (4 Marks)

Study weekly study hours as a continuous distribution.

7.1 Python code

Study hours is chosen as the continuous variable. The program plots its frequency distribution as a density histogram with a smoothed density curve (KDE) and a normal curve fitted with the sample mean (13.99) and standard deviation (4.35). The proportion of students studying **10 to 18 hours** per week is then found in two ways: by counting the data and from the normal curve.

Python code *(file: task7_continuous.py)*

```
# TASK 7 - Continuous Distribution Analysis (Study_Hours)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, gaussian_kde

df = pd.read_csv("students_clean.csv")
x = df["Study_Hours"]

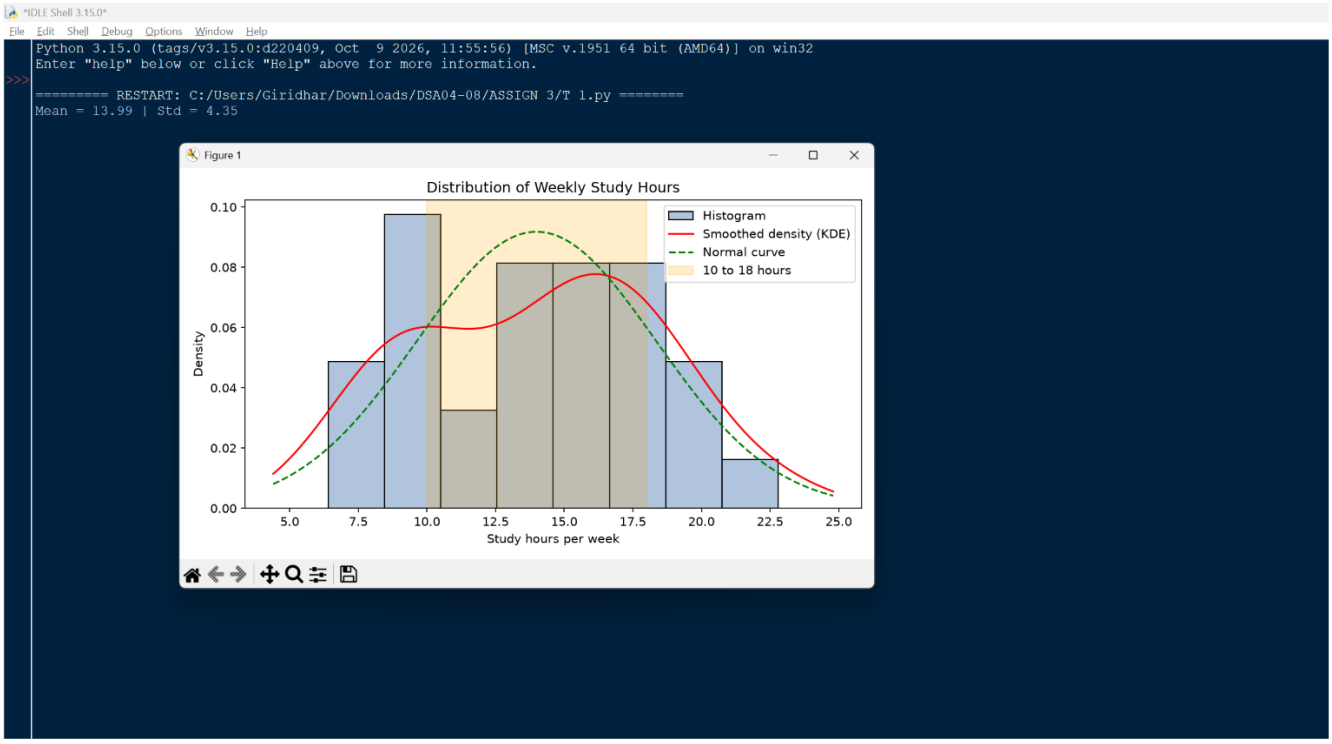
mu, sd = x.mean(), x.std()
print("Mean =", round(mu, 2), "| Std =", round(sd, 2))

# frequency distribution with density curves
grid = np.linspace(x.min() - 2, x.max() + 2, 200)
plt.figure(figsize=(8, 4.5))
plt.hist(x, bins=8, density=True, color="lightsteelblue",
         edgecolor="black", label="Histogram")
plt.plot(grid, gaussian_kde(x)(grid), color="red", label="Smoothed density (KDE)")
plt.plot(grid, norm.pdf(grid, mu, sd), color="green", linestyle="--", label="Normal curve")
plt.axvspan(10, 18, color="orange", alpha=0.2, label="10 to 18 hours")
plt.title("Distribution of Weekly Study Hours")
plt.xlabel("Study hours per week")
plt.ylabel("Density")
plt.legend()
plt.tight_layout()
plt.savefig("task7_distribution.png", dpi=120)
plt.show()

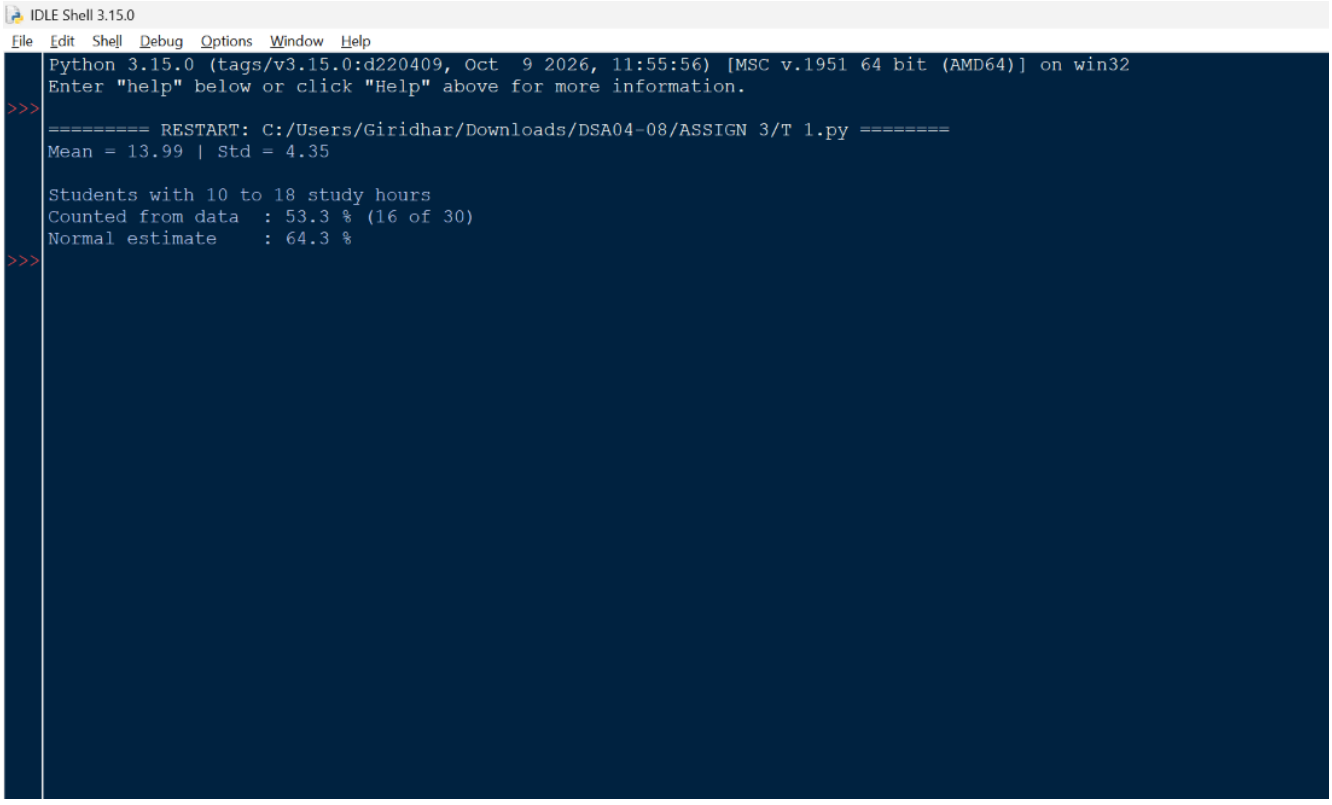
# proportion of students studying between 10 and 18 hours
lo, hi = 10, 18
inside = (x >= lo) & (x <= hi)
actual = inside.mean() * 100
normal = (norm.cdf(hi, mu, sd) - norm.cdf(lo, mu, sd)) * 100

print(f"\nStudents with {lo} to {hi} study hours")
print("Counted from data  :", round(actual, 1), "%", f"({inside.sum()} of {len(x)})")
print("Normal estimate    :", round(normal, 1), "%")
```

Output – Screenshot 8



Output – Screenshot 9



7.2 Meaning of a continuous distribution

A continuous variable can take any value within a range, not only whole numbers: a student may study 13.7 or 13.75 hours. Because there are infinitely many possible values, the probability of one exact value is essentially zero. Instead, we look at **intervals**, and the share of students in an interval is the **area under the density curve**

between its two ends. The histogram is the data version of this idea, with the bar heights scaled so that the total area is 1. In this context it describes how weekly study time is spread across the student group.

7.3 Interval result

Interval 10 to 18 hours	Result
Counted from the data	16 of 30 students = 53.3%
Estimated from the normal curve	64.3%
Students below 10 hours	8 (26.7%)
Students above 18 hours	6 (20.0%)

7.4 Interpretation

Just over half of the students (53.3%) study between 10 and 18 hours a week. The other 14 students are split between those who study less than 10 hours (8 students, about a quarter of the group) and those who study more than 18 hours (6 students). So the group is not concentrated around one study routine: a sizeable minority studies very little.

The normal curve overestimates the interval (64.3% against 53.3% observed). The plot shows why: the actual distribution is flatter than the normal curve, and the KDE has a small second hump near 8 to 9 hours, so more students sit at the low end than a bell shape predicts. With only 30 students such differences are expected, and the observed 53.3% is the more trustworthy figure for this group.

TASK 8 – Sampling and Estimation (4 Marks)

Use a sample to estimate population characteristics.

8.1 Python code

The sampling method is **simple random sampling without replacement**: 10 of the 30 students are drawn using `df.sample(n=10, random_state=7)`, so every student has the same chance of selection and the result can be repeated. Internal marks is the selected variable. The program also draws samples of 5 and 20 students to show the effect of sample size.

Python code *(file: task8_sampling.py)*

```
# TASK 8 - Sampling and Estimation (Internal_Marks)
import pandas as pd

pd.set_option("display.width", 200)

df = pd.read_csv("students_clean.csv")
col = "Internal_Marks"

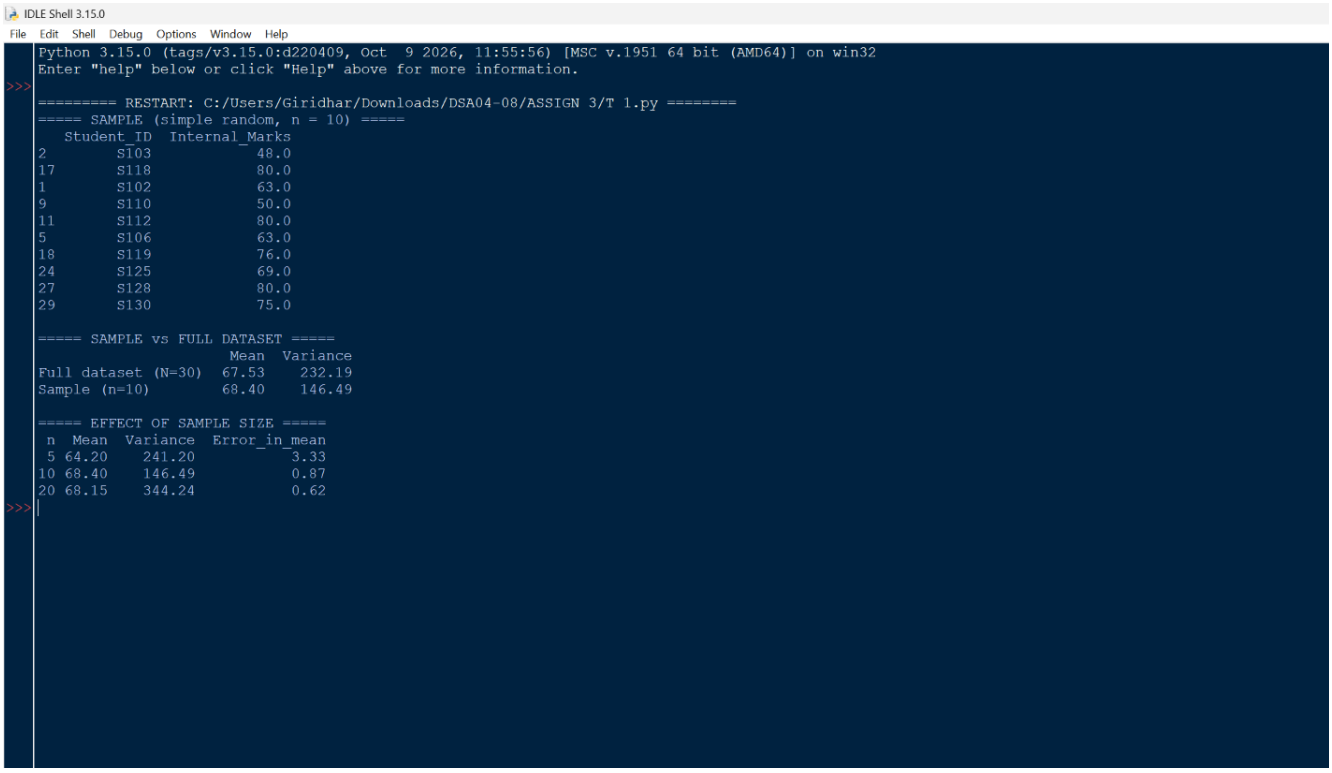
pop_mean, pop_var = df[col].mean(), df[col].var()

# simple random sample of 10 students (no replacement)
sample = df.sample(n=10, random_state=7)
print("===== SAMPLE (simple random, n = 10) =====")
print(sample[["Student_ID", col]])

table = pd.DataFrame({
    "Mean": [pop_mean, sample[col].mean()],
    "Variance": [pop_var, sample[col].var()],     # n - 1 in both cases
}, index=["Full dataset (N=30)", "Sample (n=10)"])
print("\n===== SAMPLE vs FULL DATASET =====")
print(table.round(2))

# effect of sample size
print("\n===== EFFECT OF SAMPLE SIZE =====")
rows = []
for n in [5, 10, 20]:
    s = df[col].sample(n=n, random_state=7)
    rows.append([n, s.mean(), s.var(), abs(s.mean() - pop_mean)])
size_table = pd.DataFrame(rows, columns=["n", "Mean", "Variance", "Error_in_mean"])
print(size_table.round(2).to_string(index=False))
```

Output – Screenshot 10



8.2 Calculation table

Group	Mean	Variance
Full dataset (N = 30)	67.53	232.19
Sample (n = 10)	68.40	146.49

Sample size (n)	Sample mean	Sample variance	Error in mean
5	64.20	241.20	3.33
10	68.40	146.49	0.87
20	68.15	344.24	0.62

8.3 Comparison and explanation

Comparison. The sample mean (68.40) is very close to the full-data mean (67.53), a difference of only 0.87 marks, so the sample estimates the average well. The sample variance (146.49) is clearly smaller than the full-data variance (232.19). The reason is visible in the sample: it does not contain either extreme record (S107 or S122), so it looks more uniform than the whole group.

Effect of sample size. The error in the mean falls as the sample gets larger (3.33, 0.87 and 0.62 for n = 5, 10 and 20), because a larger sample averages out chance variation. The variance does not behave as smoothly: the sample of 20 gives 344.24, higher than the true value, because it happens to contain both extreme records, while the samples of 5 and 10 contain neither. Variance is much more sensitive to a few extreme values than the mean is, so estimates of spread need larger samples.

Effect of sampling method. Simple random sampling gives every student an equal chance, so the estimates are unbiased on average. A convenience sample (for example the first 10 rows, or one department only) would risk

a biased picture, since the file is not in random order and the departments differ. When groups such as departments matter, stratified sampling (a random sample from each department) would represent them better. The sample variance uses the divisor n − 1, which corrects the tendency of small samples to underestimate spread.

TASK 9 – Covariance and Correlation Analysis (8 Marks)

Investigate the relationships among the learning and performance variables.

9.1 Python code

Covariance (`cov()`) and the Pearson correlation coefficient (`corr()`) are calculated for attendance with internal marks and study hours with internal marks. The program then builds a correlation matrix for all five numerical variables, draws it as a heatmap, and draws two scatter plots. It also repeats the correlations without the two extreme internal-marks records (S107 and S122) to see their effect.

Python code *(file: task9_cov_corr.py)*

```
# TASK 9 - Covariance and Correlation
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

df = pd.read_csv("students_clean.csv")
num = ["Attendance", "Study_Hours", "Internal_Marks", "Assignment_Score", "End_Sem_Marks"]

print("===== COVARIANCE =====")
print("Attendance vs Internal_Marks :", round(df["Attendance"].cov(df["Internal_Marks"]), 2))
print("Study_Hours vs Internal_Marks:", round(df["Study_Hours"].cov(df["Internal_Marks"]), 2))

print("\n===== PEARSON CORRELATION =====")
print("Attendance vs Internal_Marks :", round(df["Attendance"].corr(df["Internal_Marks"]), 3))
print("Study_Hours vs Internal_Marks:", round(df["Study_Hours"].corr(df["Internal_Marks"]), 3))

# same correlations again without the two extreme internal-marks records (S107, S122)
trim = df[~df["Student_ID"].isin(["S107", "S122"])]
print("\n===== WITHOUT S107 AND S122 =====")
r1 = trim["Attendance"].corr(trim["Internal_Marks"])
r2 = trim["Study_Hours"].corr(trim["Internal_Marks"])
print("Attendance vs Internal_Marks :", round(r1, 3))
print("Study_Hours vs Internal_Marks:", round(r2, 3))

corr = df[num].corr()
print("\n===== CORRELATION MATRIX =====")
print(corr.round(2))

# heatmap
fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
ax.set_xticks(range(len(num)))
ax.set_yticks(range(len(num)))
ax.set_xticklabels(num, rotation=45, ha="right")
ax.set_yticklabels(num)
for i in range(len(num)):
    for j in range(len(num)):
        ax.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center")
fig.colorbar(im)
ax.set_title("Correlation Matrix Heatmap")
plt.tight_layout()
plt.savefig("task9_heatmap.png", dpi=120)
plt.show()

# scatter plots
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
for a, c in zip(ax, ["Attendance", "Study_Hours"]):
    a.scatter(df[c], df["Internal_Marks"], color="teal")
    a.set_title(f"{c} vs Internal_Marks")
    a.set_xlabel(c)
    a.set_ylabel("Internal_Marks")
plt.tight_layout()
```

```
plt.savefig("task9_scatter.png", dpi=120)
plt.show()
```

Output – Screenshot 12

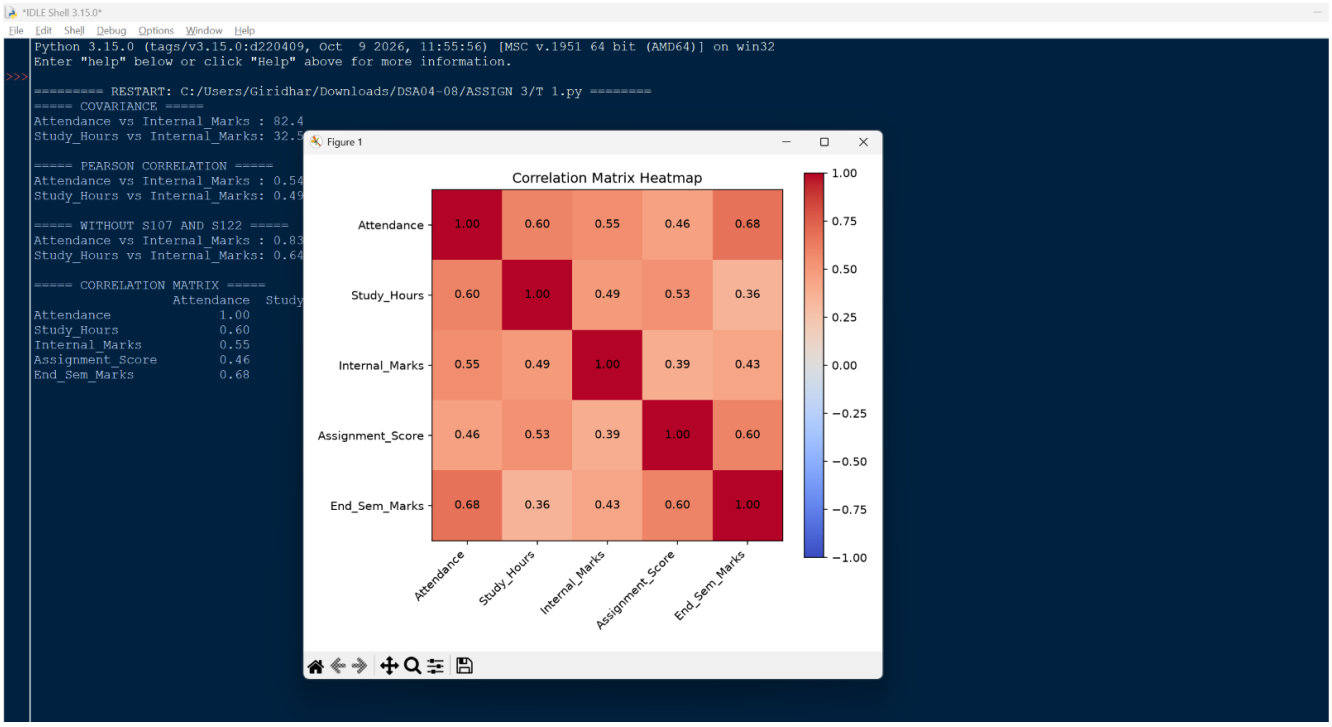
```
*IDLE Shell 3.15.0*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.15.0 (tags/v3.15.0:d220409, Oct  9 2026, 11:55:56) [MSC v.1951 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========= RESTART: C:/Users/Giridhar/Downloads/DSA04-08/ASSIGN 3/T 1.py ========
===== COVARIANCE =====
Attendance vs Internal_Marks : 82.4
Study_Hours vs Internal_Marks: 32.52

===== PEARSON CORRELATION =====
Attendance vs Internal_Marks : 0.549
Study_Hours vs Internal_Marks: 0.491

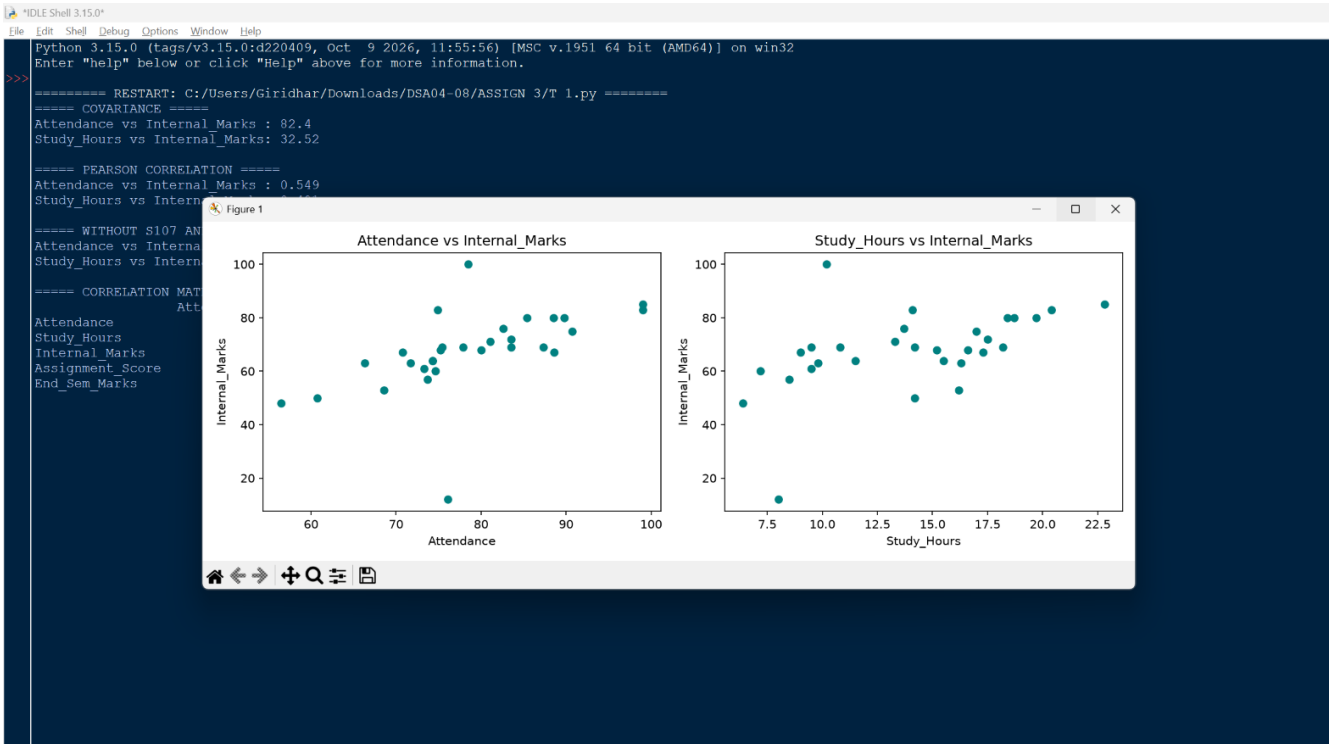
===== WITHOUT S107 AND S122 =====
Attendance vs Internal_Marks : 0.837
Study_Hours vs Internal_Marks: 0.643

===== CORRELATION MATRIX =====
                  Attendance  Study_Hours  Internal_Marks  Assignment_Score  End_Sem_Marks
Attendance              1.00         0.60            0.55              0.46           0.68
Study_Hours             0.60         1.00            0.49              0.53           0.36
Internal_Marks          0.55         0.49            1.00              0.39           0.43
Assignment_Score        0.46         0.53            0.39              1.00           0.60
End_Sem_Marks           0.68         0.36            0.43              0.60           1.00
```

Output – Screenshot 13



Output – Screenshot 14



9.2 Covariance and correlation values

Variable pair	Covariance	Pearson r	r without S107 and S122
Attendance and Internal_Marks	82.40	0.549	0.837
Study_Hours and Internal_Marks	32.52	0.491	0.643

9.3 Correlation matrix

	Attendance	Study_Hours	Internal_Marks	Assignment_Score	End_Sem_Marks
Attendance	1.00	0.60	0.55	0.46	0.68
Study_Hours	0.60	1.00	0.49	0.53	0.36
Internal_Marks	0.55	0.49	1.00	0.39	0.43
Assignment_Score	0.46	0.53	0.39	1.00	0.60
End_Sem_Marks	0.68	0.36	0.43	0.60	1.00

9.4 Interpretation of direction and strength

- **Attendance and internal marks (r = 0.549):** a moderate positive relationship. Students with higher attendance tend to have higher internal marks. The scatter plot rises from left to right, with two points (S107 and S122) clearly off the trend.
- **Study hours and internal marks (r = 0.491):** also a moderate positive relationship, slightly weaker than attendance. More weekly study goes with higher marks, but the points are more scattered.
- **Effect of the extreme records:** without S107 and S122 the correlations rise to 0.837 and 0.643, which are strong and moderate-to-strong. The two extreme marks weaken both relationships, which confirms the concern raised in Task 5.

- **Across the matrix:** the strongest link is Attendance with End_Sem_Marks (0.68), followed by Assignment_Score with End_Sem_Marks (0.60) and Attendance with Study_Hours (0.60). The weakest is Study_Hours with End_Sem_Marks (0.36). All relationships are positive; none is negative.

9.5 Covariance versus correlation, and causation

Covariance shows the direction in which two variables move together: positive here, since both covariances are above zero. Its size, however, depends on the units and scales of the variables, so 82.40 and 32.52 cannot be compared to say which relationship is stronger. **Correlation** divides the covariance by the two standard deviations, giving a unit-free value between −1 and +1 that shows both direction and strength, and can be compared across pairs.

Correlation does not establish causation. A strong correlation only shows that two variables move together. It does not show that one produces the other. Attendance and marks could both be driven by a third factor, such as motivation, interest in the subject, or health, and the relationship could partly run the other way (students who do well may be more willing to attend). Only a controlled study could show whether raising attendance would raise marks, so the results here should be read as patterns to investigate, not as proof of cause.

TASK 10 – Integrated Descriptive Statistics Case Study (7 Marks)

A department wants to find patterns in the dataset containing attendance, study hours, internal marks, assignment scores and end-semester marks. This task combines the techniques of Tasks 1 to 9 in one connected analysis, starting from the raw file.

10.1 Steps and assumptions

Step	What the program does	Assumption / method
1. Prepare	Reads students_raw.csv, removes the duplicate, fills missing values	Duplicate: first kept. Missing: column median
2. Summarise	Mean, median, standard deviation, minimum, maximum for all five variables	Sample standard deviation (n − 1)
3. Outliers	IQR rule applied to every numerical variable	Bounds Q1 − 1.5 IQR and Q3 + 1.5 IQR
4. Shape	Skewness and excess kurtosis of internal and end-semester marks	SciPy functions
5. Relationships	Correlation of every variable with end-semester marks; covariance of two pairs	Pearson correlation
6. Groups	Students grouped into attendance bands; students needing support listed	Bands: below 75, 75 to 85, above 85. Support list: attendance below 75 and end-sem marks below 62 (thresholds chosen for illustration)
7. Plots	Scatter plot with trend line; box plots by attendance band	Matplotlib

10.2 Python code (connected workflow)

Python code *(file: task10_case_study.py)*

```
# TASK 10 - Integrated Descriptive Statistics Case Study
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import skew, kurtosis

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

# ---- 1. prepare and inspect ----
df = pd.read_csv("students_raw.csv")
print("Raw shape:", df.shape)
print("Duplicates:", df.duplicated().sum(), "| missing:", df.isnull().sum().sum())
df = df.drop_duplicates()
for c in ["Study_Hours", "Assignment_Score"]:
    df[c] = df[c].fillna(df[c].median())
print("Clean shape:", df.shape, "| missing left:", df.isnull().sum().sum())

num = ["Attendance", "Study_Hours", "Internal_Marks", "Assignment_Score", "End_Sem_Marks"]

# ---- 2. summary statistics ----
summary = pd.DataFrame({
    "Mean": df[num].mean(), "Median": df[num].median(),
    "Std_Dev": df[num].std(), "Min": df[num].min(), "Max": df[num].max(),
})
print("\n===== SUMMARY STATISTICS =====")
print(summary.round(2))

# ---- 3. outliers (IQR) ----
print("\n===== OUTLIERS BY IQR RULE =====")
```

```
for c in num:
    q1, q3 = df[c].quantile([0.25, 0.75])
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    out = df.loc[(df[c] < low) | (df[c] > high), ["Student_ID", c]]
    print(f"{c:17s} bounds [{low:6.2f}, {high:6.2f}] -> {len(out)} outlier(s)",
          out.values.tolist())

# ---- 4. skewness and kurtosis ----
print("\n===== SKEWNESS AND KURTOSIS =====")
for c in ["Internal_Marks", "End_Sem_Marks"]:
    print(f"{c:15s} skew = {skew(df[c]):6.3f} | excess kurtosis = {kurtosis(df[c]):6.3f}")

# ---- 5. correlation ----
print("\n===== CORRELATION WITH End_Sem_Marks =====")
print(df[num].corr()["End_Sem_Marks"].drop("End_Sem_Marks").round(3))
cov1 = df["Attendance"].cov(df["End_Sem_Marks"])
cov2 = df["Study_Hours"].cov(df["Internal_Marks"])
print("\nCovariance  Attendance vs End_Sem_Marks:", round(cov1, 2))
print("Covariance  Study_Hours vs Internal_Marks:", round(cov2, 2))

# ---- 6. attendance bands ----
df["Attendance_Band"] = pd.cut(df["Attendance"], [0, 75, 85, 100],
                               labels=["Below 75", "75 to 85", "Above 85"], right=False)
band = df.groupby("Attendance_Band", observed=True).agg(
    Students=("Student_ID", "count"),
    Avg_Internal=("Internal_Marks", "mean"),
    Avg_End_Sem=("End_Sem_Marks", "mean"),
).round(2)
print("\n===== PERFORMANCE BY ATTENDANCE BAND =====")
print(band)

risk = df[(df["Attendance"] < 75) & (df["End_Sem_Marks"] < 62)]
print("\n===== STUDENTS NEEDING SUPPORT (attendance < 75 and end-sem < 62) =====")
print(risk[["Student_ID", "Department", "Attendance", "Study_Hours", "End_Sem_Marks"]])

# ---- plots ----
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))

ax[0].scatter(df["Attendance"], df["End_Sem_Marks"], color="teal")
m, b = np.polyfit(df["Attendance"], df["End_Sem_Marks"], 1)
xs = np.linspace(df["Attendance"].min(), df["Attendance"].max(), 50)
ax[0].plot(xs, m * xs + b, color="red", label=f"Trend line (slope = {m:.2f})")
ax[0].set_title("Attendance vs End-Semester Marks")
ax[0].set_xlabel("Attendance (%)")
ax[0].set_ylabel("End-semester marks")
ax[0].legend()

groups = [df.loc[df["Attendance_Band"] == g, "End_Sem_Marks"] for g in band.index]
ax[1].boxplot(groups)
ax[1].set_xticklabels(band.index)
ax[1].set_title("End-Semester Marks by Attendance Band")
ax[1].set_xlabel("Attendance band (%)")
ax[1].set_ylabel("End-semester marks")

plt.tight_layout()
plt.savefig("task10_plots.png", dpi=120)
plt.show()
```

Output – Screenshot 15

```
*IDLE Shell 3.15.0*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.15.0 (tags/v3.15.0:d220409, Oct  9 2026, 11:55:56) [MSC v.1951 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>

========= RESTART: C:/Users/Giridhar/Downloads/DSA04-08/ASSIGN 3/T 1.py ========
Raw shape: (31, 7)
Duplicates: 1 | missing: 2
Clean shape: (30, 7) | missing left: 0

===== SUMMARY STATISTICS =====
                    Mean  Median  Std_Dev   Min    Max
Attendance         78.73    77.0     9.85  56.5   99.0
Study_Hours        13.99    14.2     4.35   6.4   22.8
Internal_Marks     67.53    68.5    15.24  12.0  100.0
Assignment_Score   10.64    10.6     2.11   6.4   15.0
End_Sem_Marks      66.90    66.5     8.52  46.0   81.0

===== OUTLIERS BY IQR RULE =====
Attendance         bounds [ 57.24, 101.54] -> 1 outlier(s) [['S103', 56.5]]
Study_Hours        bounds [ -1.09,  28.21] -> 0 outlier(s) []
Internal_Marks     bounds [ 43.88,  94.88] -> 2 outlier(s) [['S107', 12.0], ['S122', 100.0]]
Assignment_Score   bounds [  5.66,  15.96] -> 0 outlier(s) []
End_Sem_Marks      bounds [ 42.50,  92.50] -> 0 outlier(s) []

===== SKEWNESS AND KURTOSIS =====
Internal_Marks   skew = -1.342 | excess kurtosis =  4.420
End_Sem_Marks    skew = -0.344 | excess kurtosis = -0.350

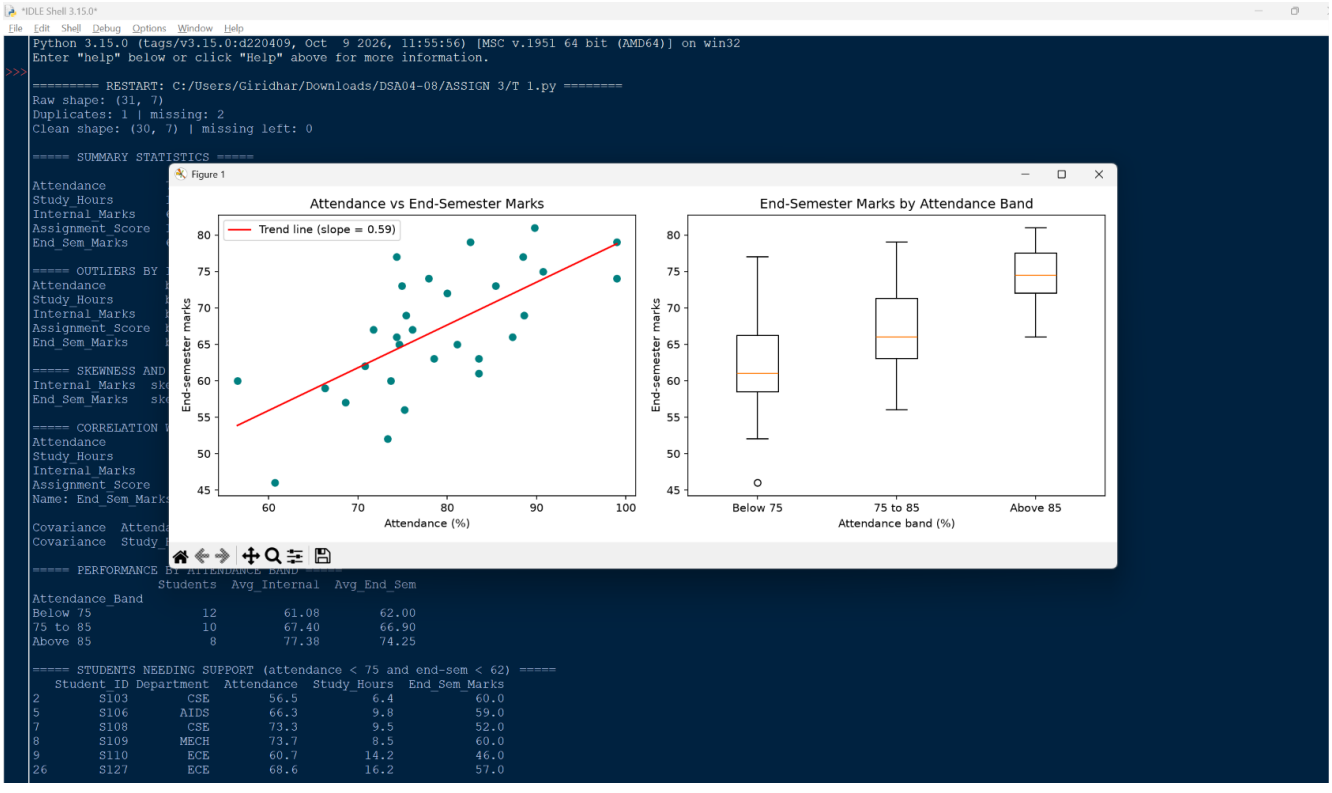
===== CORRELATION WITH End_Sem_Marks =====
Attendance          0.678
Study_Hours         0.358
Internal_Marks      0.434
Assignment_Score    0.597
Name: End_Sem_Marks, dtype: float64

Covariance  Attendance vs End_Sem_Marks: 56.86
Covariance  Study_Hours vs Internal_Marks: 32.52

===== PERFORMANCE BY ATTENDANCE BAND =====
                Students  Avg_Internal  Avg_End_Sem
Attendance_Band
Below 75              12         61.08        62.00
75 to 85              10         67.40        66.90
Above 85               8         77.38        74.25

===== STUDENTS NEEDING SUPPORT (attendance < 75 and end-sem < 62) =====
   Student_ID Department  Attendance  Study_Hours  End_Sem_Marks
2        S103        CSE        56.5          6.4           60.0
5        S106       AIDS        66.3          9.8           59.0
7        S108        CSE        73.3          9.5           52.0
8        S109       MECH        73.7          8.5           60.0
9        S110        ECE        60.7         14.2           46.0
26       S127        ECE        68.6         16.2           57.0
```

Output – Screenshot 16



10.3 Results

Summary statistics (30 students)

Variable	Mean	Median	Std. dev.	Min	Max
Attendance	78.73	77.00	9.85	56.5	99.0
Study_Hours	13.99	14.20	4.35	6.4	22.8
Internal_Marks	67.53	68.50	15.24	12.0	100.0
Assignment_Score	10.64	10.60	2.11	6.4	15.0
End_Sem_Marks	66.90	66.50	8.52	46.0	81.0

Outliers by the IQR rule

Variable	Bounds	Outliers found
Attendance	57.24 to 101.54	1 (S103: 56.5)
Study_Hours	−1.09 to 28.21	None
Internal_Marks	43.88 to 94.88	2 (S107: 12, S122: 100)
Assignment_Score	5.66 to 15.96	None
End_Sem_Marks	42.50 to 92.50	None

Skewness, kurtosis and correlation with end-semester marks

Measure	Internal_Marks	End_Sem_Marks
Skewness	−1.342	−0.344
Excess kurtosis	4.420	−0.350

Variable	Correlation with End_Sem_Marks
Attendance	0.678
Assignment_Score	0.597
Internal_Marks	0.434
Study_Hours	0.358

Covariance of Attendance with End_Sem_Marks is 56.86, and of Study_Hours with Internal_Marks is 32.52; both are positive.

Performance by attendance band

Attendance band (%)	Students	Avg. internal marks	Avg. end-sem marks
Below 75	12	61.08	62.00
75 to 85	10	67.40	66.90
Above 85	8	77.38	74.25

Students needing support (attendance below 75 and end-sem marks below 62)

Student	Department	Attendance	Study hours	End-sem marks
S103	CSE	56.5	6.4	60
S106	AIDS	66.3	9.8	59
S108	CSE	73.3	9.5	52
S109	MECH	73.7	8.5	60
S110	ECE	60.7	14.2*	46
S127	ECE	68.6	16.2	57

**S110's study hours was missing in the raw file and was filled with the median (14.2).*

10.4 Interpretation of the plots

The scatter plot shows a clear upward pattern: students with higher attendance tend to score higher in the end-semester examination, and the trend line rises by about 0.59 marks for every extra percentage point of attendance. The box plots make the same point by group. The median end-semester mark increases from about 61 in the below-75% band to about 66 in the middle band and about 74.5 in the above-85% band, and the lower band also contains the lowest score (46, student S110).

10.5 Findings

- **Finding 1 – Attendance is the variable most closely linked to end-semester results.** Its correlation with end-semester marks is 0.678, the highest of all variables. Students attending more than 85% score 74.25 on average, against 62.00 for those below 75%, a gap of 12.25 marks. Because this is a correlation, it shows association, not proof that attendance alone causes the difference.
- **Finding 2 – Internal marks contain two extreme records that distort the summaries.** S107 (12) and S122 (100) are the only outliers among the internal marks. They push the skewness to −1.342 and the kurtosis to 4.420, and they weaken the correlation of internal marks with attendance (0.549, rising to 0.837 without them). End-semester marks have no outliers and are almost symmetric (skewness −0.344, kurtosis −0.350), so the end-semester results are more stable.
- **Finding 3 – Assignment performance also goes with final results.** Assignment scores correlate 0.597 with end-semester marks, higher than study hours (0.358) or internal marks (0.434). Regular assignment work may be a useful early indicator of final performance.

10.6 Recommended academic actions

- **Action 1 – Introduce an early-warning and mentoring system for low attendance.** Students whose attendance falls below 75% should be flagged and given a mentor. The six students listed above (S103, S106, S108, S109, S110 and S127) combine low attendance with end-semester marks below 62 and should be contacted first, with remedial sessions where needed. This follows from Finding 1.
- **Action 2 – Verify the extreme internal-mark records and use robust measures in reporting.** The department should check the entries for S107 (12) and S122 (100) against the original answer scripts to confirm whether they are real or data-entry errors, and should report the median alongside the mean for internal marks until this is done. This follows from Finding 2; Finding 3 also supports monitoring assignment submissions as an additional warning sign.

Limitation. The dataset is a small hypothetical sample of 30 students, so these results illustrate the method and should not be taken as conclusions about a real student group.

OVERALL CONCLUSION

This assignment applied descriptive statistics and exploratory data analysis to a student-performance dataset using Python. Data preparation (Task 1) removed one duplicate and two missing values so that the later measures could be trusted. Summary measures and measures of spread (Tasks 2 and 3) showed that internal marks are the most variable variable, and the median describes them better than the mean. Histograms and box plots (Task 4) and the IQR rule (Task 5) showed that two records, 12 and 100, are responsible for much of this variability; capping them reduced the standard deviation by about 24% and removed the strong skew (Task 6).

Study hours was examined as a continuous variable (Task 7), sampling showed that a random sample estimates the mean well but the variance only roughly (Task 8), and correlation analysis showed that attendance is positively and moderately to strongly related to marks (Task 9). The integrated case study (Task 10) combined these steps and led to two practical recommendations: monitor low attendance early, and verify the extreme internal-mark records. Throughout, the results are described as relationships in the data, not as proof of cause.

REFERENCES / LEARNING RESOURCES

- Pandas documentation – https://pandas.pydata.org/docs/
- NumPy documentation – https://numpy.org/doc/
- SciPy statistics module (scipy.stats) – https://docs.scipy.org/doc/scipy/reference/stats.html
- Matplotlib documentation – https://matplotlib.org/stable/
- DSA04 – Fundamentals of Data Science, Unit notes and Assignment 3 – CO3 brief, SIMATS Engineering.